

ADAF-RedTeam

Capability Certification How-To

Use docs/CERTIFICATION_RUNBOOK.md + a disposable lab · Fill evidence templates · Get reviewer sign-off · Promote lab_certified=True one capability at a time

What this document is

A step-by-step operator guide for certifying Tier A (read/metadata) capabilities against a disposable Active Directory lab, assembling the evidence package, obtaining independent review, and opening a one-capability promotion PR. It consolidates docs/CERTIFICATION.md and docs/CERTIFICATION_RUNBOOK.md.

What certification is NOT

- Not approval to run against production — engagement authorization remains separate.
- Not a relaxation of gates — authorization, containment, cleanup latch, and redaction still run.
- Not transferable — each capability is certified on its own.
- Not a blanket flag flip — never set multiple capabilities to True in one unreviewed change.

1. Mental model

Every capability ships with lab_certified=False. Live runs without a completed certification are stamped **UNVALIDATED**. Certification means: the live primitive was proven correct, bounded, and (for higher tiers) reversible **in a disposable lab**, with a complete evidence package and an independent reviewer who did not write the primitive.

lab_certified=True only removes the UNVALIDATED stamp. It never disables the runtime authorization gate, containment probe, cleanup latch, or SecretVault.

1.1 Roles (no self-certification)

Role	Responsibility
Certification owner	Implements/runs the live path, assembles evidence package, owns the ticket.
Independent reviewer	Did not write the primitive; verifies evidence, tests, and plan() bounds; signs off.
Second approver	Required only for Tier E (destructive), e.g. zerologon-reset.

1.2 Tiers (summary)

Tier	Examples	Extra bar
A — read/metadata	asrep, kerberoast, dcsync-rights, laps, gmsa, zerologon-detection	No write; no secret export; pcap proof
B — reversible write	rbcd-write, shadow-cred, exec-proof-svcctl	Restore + cleanup latch proven
C — durable	adcs-esc1	Honest durableResidue; lab rebuild

D – purple team	adversary-emulation-evasion	Detection block matches real SOC/EDR
E – destructive	zerologon-reset	Reset-then-restore; two-person rule

This how-to focuses on **Tier A** (first wave). Higher tiers reuse the same process with the extra bars above.

2. Non-negotiable preconditions

Stop if any of these are false

Certification work happens **only** in a disposable lab you can rebuild from scratch. No route to production. No shared trust with production. Snapshot or rebuild capability required. If a primitive cannot be made safe and reversible in that lab, it stays `lab_certified=False` indefinitely — that is an acceptable outcome.

- Lab isolated (no production route / shared trust); snapshot- or rebuild-capable.
- Certification engagement JSON authorizes the exact capability and targets, with `labAddressRanges` / `labResolvedAddresses` that pass containment for the lab and fail for anything else.
- Work logged against a certification ticket with a named owner.
- Operator host can capture packets (required for Zerologon “no password-set” proof).
- Editable install: `python -m pip install -e ".[ldap,kerberos,dev]"`

3. Lab inventory & .env.lab

Create a local, **untracked** file (never commit secrets, tickets, or validation results). Example only — replace every value with your lab's real identifiers.

```
# .env.lab
export ADAF_RT_LAB=1
export ADAF_RT_LAB_DOMAIN=corp.contoso.test
export ADAF_RT_LAB_DC=dc01.corp.contoso.test
export ADAF_RT_LAB_SOURCE_ADDR=10.10.0.50
export ADAF_RT_LAB_BIND_USER='CORP\certuser'
# export ADAF_RT_LAB_BIND_PASSWORD='...' # only if not using Kerberos ccache

# AS-REP (positive + negative accounts)
export ADAF_RT_LAB_ASREP_ROASTABLE_USER=svc-nopreauth
export ADAF_RT_LAB_ASREP_PREAUTH_USER=normaluser

# Kerberoast
export ADAF_RT_LAB_KERBEROAST_SPN=MSSQLSvc/db01.corp.contoso.test
export ADAF_RT_LAB_KERBEROAST_MISSING_SPN=http/does-not-exist.corp.contoso.test
export KRB5CCNAME=/tmp/lab.ccache

# DCSync / LAPS / gMSA principal under test
export ADAF_RT_LAB_TARGET_PRINCIPAL='S-1-5-21-...-512'
export ADAF_RT_LAB_EXPECTED=Confirmed

# LAPS computer object
export ADAF_RT_LAB_LAPS_COMPUTER_DN='CN=PC01,CN=Computers,DC=corp,DC=contoso,DC=test'

# gMSA object
export ADAF_RT_LAB_GMSA_DN='CN=gmsa-web,CN=Managed Service Accounts,DC=corp,DC=contoso,DC=test'

# Zerologon
export ADAF_RT_LAB_ZEROLOGON_TARGET=DC01

# Discovery
export ADAF_RT_LAB_QUOTA_EXPECTED=Confirmed
export ADAF_RT_LAB_PRIV_GROUP_DN='CN=Domain Admins,CN=Users,DC=corp,DC=contoso,DC=test'
export ADAF_RT_LAB_PRIV_EXPECTED=Confirmed
```

Load the file for the current shell:

```
set -a && source .env.lab && set +a
```

3.1 Lab object preparation (Tier A examples)

AS-REP positive case — set DONT_REQUIRE_PREAUTH:

```
Set-ADAccountControl -Identity svc-nopreauth -DoesNotRequirePreAuth $true
Get-ADUser svc-nopreauth -Properties DoesNotRequirePreAuth |
  Select-Object DoesNotRequirePreAuth
```

Kerberoast — ensure a registered SPN exists and obtain a TGT:

```
# Linux example
kinit certuser@CORP.CONTOSO.TEST
export KRB5CCNAME=/tmp/lab.ccache
klist
```

LAPS: ensure the computer has a LAPS attribute ACL you can reason about (reader vs non-reader principals).
gMSA: ensure PrincipalsAllowedToRetrieveManagedPassword includes (or excludes) the principal under test. Never retrieve the password values themselves during certification.

4. Capability matrix (Tier A first wave)

Capability ID	Cert test	Evidence template
asrep-roast-validation	tests/test_certification_asrep_roast.py	docs/certifications/asrep-roast-validation.md
kerberoast-validation	tests/test_certification_kerberoast.py	docs/certifications/kerberoast-validation.md
dcsync-rights-validation	tests/test_certification_dcsync_rights.py	docs/certifications/dcsync-rights-validation.md
zerologon-detection	tests/test_certification_zerologon.py	docs/certifications/zerologon-detection.md
laps-read-authorization	tests/test_certification_laps_read.py	docs/certifications/laps-read-authorization.md
gmsa-read-authorization	tests/test_certification_gmsa_read.py	docs/certifications/gmsa-read-authorization.md
machine-account-quota-check	tests/test_certification_discovery.py	Fill after first lab run
privileged-group-inventory	tests/test_certification_discovery.py	Fill after first lab run

5. Universal checklist (every capability)

A capability may be promoted only when **all** of the following are true and evidenced (from docs/CERTIFICATION.md §1):

- 1 **Live primitive implemented and reviewed** in probes/*.py (or LDAP source), bounded to exactly what `plan()` describes.
- 2 **Analyzer/orchestration unchanged.** Offline unit tests still pass; certification adds the live path only.
- 3 **Redaction proven.** Grep every artifact: no password, hash, ticket, key, PFX, or LAPS value. Secrets reach SecretVault handles only.
- 4 **Authorization gate honored.** Wrong source / target / technique refused.
- 5 **Idempotent, bounded, interval-respecting.** Second run does not compound state; bounds match `plan()`.
- 6 **Evidence package complete** (Section 7) and attached to the certification ticket.
- 7 **Independent review** sign-off recorded (owner ≠ reviewer).

Only then: set `lab_certified=True` for **that one** capability in `adaf_redteam/capabilities/registry.py`, in the **same PR** as the evidence and any cert-test assertion updates.

5.1 Tier A extra proofs

- **No state change:** pcap / DC audit shows only reads or bounded metadata requests.
- **No secret export:** AS-REP/Kerberoast cipher never written; DCSync issues no DRSUAPI replication; LAPS/GMSA never read password attributes.
- **Zerologon-detection:** probe stops before `NetrServerPasswordSet2`; pcap shows zero password-set calls; DC machine-account password unchanged after the run.

6. Execution procedure

6.1 Baseline offline suite

Before any live lab run, confirm the offline suite is green. Offline `--fixture` runs exercise analyzers and the bridge; they do **not** count as certification.

```
pytest -q
```

Optional offline smoke of a single capability:

```
adaf-redteam run \
  --engagement examples/engagement.example.json \
  --capability asrep-roast-validation \
  --source-address 192.0.2.25 \
  --fixture examples/offline-full-fixture.example.json \
  --finding-id F-OFFLINE000000001 \
  --control-id ADAF-OFFLINE \
  --out ./out-offline
```

6.2 Recommended live run order

```
# After: set -a && source .env.lab && set +a

# Kerberos metadata
pytest tests/test_certification_asrep_roast.py -v
pytest tests/test_certification_kerberoast.py -v

# LDAP rights / membership
pytest tests/test_certification_dcsync_rights.py -v
pytest tests/test_certification_laps_read.py -v
pytest tests/test_certification_gmsa_read.py -v

# Discovery
pytest tests/test_certification_discovery.py -v

# Zerologon – start packet capture first
pytest tests/test_certification_zerologon.py -v
```

6.3 Per-capability loop (do one at a time)

- 1 Confirm lab objects and env vars for this capability only.
- 2 Start capture if required (Zerologon).
- 3 Run the matching tests/test_certification_*.py file.
- 4 On green: copy validation-result.json and any journal into the evidence package location for the ticket.
- 5 Run the redaction grep (Section 7.2).
- 6 Fill every ■ in the capability's docs/certifications/<id>.md template.
- 7 Request independent reviewer sign-off (Section 8).
- 8 Open the promotion PR (Section 9) for **this capability only**.
- 9 Only after merge: move to the next capability.

6.4 Gate negative tests (record in evidence)

- Wrong source address → gate refuses (expect non-zero exit / ADAF-RT-E102).
- Target not on engagement list → refused (ADAF-RT-E103).
- Wrong ATT&CK technique → refused (ADAF-RT-E104).
- Live run without ADAF_RT_LAB=1 while uncertified → refused / NOT CERTIFIED path.
- Containment: engagement with non-lab labResolvedAddresses must fail for state-changing caps (practice this pattern even on Tier A engagements).

7. Evidence package

Attach the following to the certification ticket for **each** capability (from CERTIFICATION.md §3):

- 1 Redacted certification engagement file + containment probe record (lab address verified; non-lab address refused where applicable).
- 2 Read-only attestation (Tier A) or before/after state (Tier B/C/E).
- 3 Redaction scan result over all produced artifacts (must be clean).
- 4 validation-result.json from the live run + transaction journal if present.
- 5 Tier B/C/E only: forced-failure run showing cleanup latch engagement.
- 6 Tier D only: blue-team telemetry corroborating the detection block.
- 7 Certification test path + green CI link for offline suite.
- 8 Reviewer sign-off (and second approver for Tier E).

7.1 Filling the evidence template

Open the matching file under docs/certifications/. Replace every _____ placeholder. Change ■ to ■ only when the artifact exists and has been reviewed. Do not mark complete from memory.

Typical Tier A sections to complete:

- Lab identifier, snapshot/rebuild note, ticket ID, owner, reviewer names.
- Lab accounts / DNS / SIDs used (positive and negative cases where applicable).
- Pcap filename/hash and observed message counts (AS-REQ only, no password-set, etc.).
- Verdict / proofClass match for positive and negative cases.
- Redaction scan command output.
- Gate negative-test results.
- Idempotency note (second run, no compounded state).
- Sign-off table with dates.

7.2 Redaction scan commands

Generic (adjust patterns per capability evidence template):

```
grep -Eri 'password|nthash|ntlm|krbtgt:|-----BEGIN|cipher:|\$krb5asrep\$|\$krb5tgs\$|crackable|asrep-hash' <out>/ || echo c
```

Zerologon-specific:

```
grep -Eri 'passwordset|netrserverpasswordset|nthash|-----BEGIN' <out>/ || echo clean
```

LAPS / gMSA-specific reminders:

```
grep -Eri 'ms-mcs-admpwd:|managedpassword|msds-managedpassword|cleartext' <out>/ || echo clean
```

7.3 Zerologon pcap expectations

- Expect Netlogon challenge/authenticate attempts only.
- Count of NetrServerPasswordSet2 (or equivalent password-set) must be **0**.
- Confirm DC machine-account password / secure channel unchanged after the run.
- Record pcap hash and filter notes in docs/certifications/zerologon-detection.md.

8. Independent reviewer sign-off

The reviewer must be a different person from the certification owner. They verify:

- Evidence artifacts exist and match the claims in the template.
- Live bounds match plan() (one request, no extra targets, no secret export).

- Redaction scan is clean.
- Gate negatives behave as documented.
- For Tier A: no state change on the wire / audit log.
- Cert test is lab-gated (ADAF_RT_LAB=1) and does not run in normal CI.

Record name, date, and notes in the template's sign-off table. Without this row completed, do not open a promotion PR.

9. Mechanical promotion PR (one capability)

Same PR only

Set `lab_certified=True` for **exactly one** capability ID in `registry.py` in the **same PR** that carries the completed evidence updates and removes the UNVALIDATED-stamp assertion from that capability's certification test. Never flip the flag in a PR that does not carry evidence.

9.1 Checklist for the PR

- 1 Branch name example: `cert/asrep-roast-validation`
- 2 Edit `adaf_redteam/capabilities/registry.py`: only the target descriptor's `lab_certified=True`.
- 3 Edit the matching `tests/test_certification_*.py`: remove or invert the assertion that requires an UNVALIDATED stamp.
- 4 Commit filled evidence markdown under `docs/certifications/` (no secrets, no raw tickets, no password material).
- 5 PR description links the ticket, lists evidence items, names owner and reviewer.
- 6 CI offline suite green.
- 7 Merge only after review.

9.2 Example registry change (illustrative)

```
# In adaf_redteam/capabilities/registry.py – ONE descriptor only
CapabilityDescriptor(
    "asrep-roast-validation",
    "AS-REP roasting metadata",
    "kerberos",
    "Executable",
    "T1558.004",
    adapter=AsrepRoastCapability,
    lab_certified=True, # was False; flip only with evidence in this PR
),
```

9.3 Example cert-test assertion update

While uncertified, tests assert that results still carry UNVALIDATED. After promotion, delete that assertion (as noted in the cert test comments) so the suite matches the new flag.

```
# Remove this once lab_certified=True for this capability:
# assert any("UNVALIDATED" in a for a in doc["proof"]["assertions"])
```

10. De-certification

Set `lab_certified` back to `False` immediately when:

- The live primitive, its bounds, `plan()`, or cleanup changes.
- A run fails to restore and latches in a way not explained by the evidence.
- The lab no longer represents the primitive's behavior (dependency/protocol change).
- Any redaction leak is found for any capability (audit them all).

De-certification is cheap and expected. Prefer it over debating edge cases.

11. Worked example: asrep-roast-validation

11.1 Prepare

- 1 Snapshot the lab DC.
- 2 Create roastable user with `DONT_REQUIRE_PREAUTH` and a normal preauth-required user.
- 3 Fill `.env.lab` AS-REP variables; set `ADAF_RT_LAB=1`.
- 4 Open a certification ticket; name the owner.

11.2 Run

```
set -a && source .env.lab && set +a
pytest -q
pytest tests/test_certification_asrep_roast.py -v
```

11.3 Evidence

- 1 Save both runs' `validation-result.json` files.
- 2 Run redaction grep; paste "clean" into the template.
- 3 Attach pcap / note one AS-REQ per run, zero TGS follow-ons.
- 4 Record gate negatives (wrong source, wrong technique).
- 5 Complete `docs/certifications/asrep-roast-validation.md`.
- 6 Independent reviewer signs the table.

11.4 Promote

- 1 Branch `cert/asrep-roast-validation`.
- 2 Flip only that descriptor to `lab_certified=True`.
- 3 Remove `UNVALIDATED` assertion in `tests/test_certification_asrep_roast.py`.
- 4 Commit evidence markdown; open PR; merge after review.
- 5 Do not start kerberoast promotion until this PR is merged and the ticket closed.

12. Troubleshooting

Symptom	Likely cause	What to do
Cert test skipped	<code>ADAF_RT_LAB</code> ≠1 or missing env	Source <code>.env.lab</code> ; check <code>REQUIRED_ENV</code> in the test file
LDAP bind failed	Wrong user / no ccache / not LDAPS	Set <code>BIND_USER</code> ; kinit; confirm port 636
AS-REP always NotExploitable	Preauth still required	Verify <code>DoesNotRequirePreAuth</code> on positive user
Kerberoast fails TGT	<code>KRB5CCNAME</code> empty/expired	kinit; export <code>KRB5CCNAME</code> ; klist

UNVALIDATED missing after flip	Flag flipped without test edit	Never flip without removing the assertion in the same PR
Redaction grep dirty	Secret leaked to JSON/journal	Do not promote; fix probe; de-certify peers if needed

13. Non-goals (operator reminders)

- Do not set `lab_certified=True` from the runbook alone.
- Do not run certification tests against production or shared non-disposable DCs.
- Do not commit `.env`, `lab`, tickets, hashes, TGTs, or validation results with secrets.
- Do not implement half-correct live Tier B/C writers just to flip a flag — leave `NotImplementedError` until lab-backed.
- Do not treat certification as production authorization.

Appendix A — Source documents in the repo

- `docs/CERTIFICATION.md` — policy gate, tiers, evidence, promotion, de-certification, roles.
- `docs/CERTIFICATION_RUNBOOK.md` — env file, run order, offline alternative.
- `docs/certifications/*.md` — per-capability evidence templates.
- `tests/test_certification_*.py` — lab-gated tests (`ADAF_RT_LAB=1`).
- `adaf_redteam/capabilities/registry.py` — sole place to flip `lab_certified`.
- `adaf_redteam/lab_env.py` — lab DC / bind / object-DN helpers.

Appendix B — Quick reference: promote one capability

```

1. Disposable lab ready + ticket open
2. source .env.lab # ADAF_RT_LAB=1
3. pytest -q
4. pytest tests/test_certification_<capability>.py -v
5. Redaction grep → clean
6. Fill docs/certifications/<capability>.md
7. Independent reviewer signs
8. PR: registry lab_certified=True (one id) + remove UNVALIDATED assert + evidence
9. Merge → next capability

```

Final rule

If the evidence is incomplete, the reviewer is the same person as the owner, or the PR flips more than one capability without per-id evidence, **do not merge**. Certification is slow on purpose.